PROJECT BRIEF

TrustNav

Live trust monitoring for satellite positions: detection, explanation and response to GNSS jamming, spoofing and forgery, for vehicle and drone fleets.

99.3 %	96.1 %	0.44 %	8 km → 29 m
precision	recall	false alarms	worst lie, before and after the response layer

What it is. Every second, for each vehicle, TrustNav scores whether its satellite position can be believed (0 to 100), explains why in plain words, and refuses positions it does not believe, tracking the vehicle by its odometer instead. Three kinds of evidence are fused: physical rules, an anomaly model trained on normal driving only, and Galileo's own digital signatures (OSNMA).

Where it is. Live demo at trustnav.vercel.app. Source, tests and this document at github.com/domail2karandeshmukh-cyber/trustnav. Our attacks are simulated in software and the real ones are recordings made by others; nothing is transmitted.

On real attacks. Replayed on public recordings from Jammertest 2024: the first version missed a phone captured by a real spoofer for 21 min 49 s; a sixteenth rule on receiver time now refuses every captured fix from the first second.

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers
Karan Deshmukh, Sudheer Reddy Jogi, Niranjan Satheesh Thandassery, Venkata Pavan Kumar Sanam
Tutor: Sofiane Hamrioui · Engine 2.2.0 · 11 November 2026

1 Summary

What. TrustNav is live trust monitoring for satellite positions. For every vehicle and drone in a fleet, once per second, it answers *"can this position be believed?"* with a score from 0 to 100, says *why* in plain words, and, since version 2.1, *acts*: a position it does not believe is refused and replaced by an estimate from the odometer along the planned route, with a recommended action for the operator.

Why. Jamming (drowning the satellite signal) and spoofing (faking it) have become everyday events in Europe. The equipment costs 30 to 300 EUR. Ordinary fleet software shows whatever position the receiver reports and never asks whether it is true. Existing detection products either live inside one receiver and output a flag with no explanation, or are fixed monitoring stations for sites; none gives a fleet operator a per-vehicle, explained, actionable trust score (see COMPARISON.md (section 7)).

How. Three kinds of evidence, fused:

1. **Rules.** Sixteen checks (a van cannot move 8 km in a second; a signal cannot be stronger and more uniform than real satellites; the receiver's time cannot jump against the clock of the device it is plugged into) with

named thresholds, so every alert can be verified by hand.

2. **Model.** An Isolation Forest and a per-feature tail detector trained on *normal* driving only, which notice the slow, careful attacks the rules miss (a van 12 m off its road).
3. **Galileo OSNMA.** The satellites' own digital signatures (operational since July 2025): a failed signature is strong evidence of forgery.

Rules and model are combined by noisy-OR (either witness can raise the alarm); trust = 100 minus the combined risk. An incident opens after three seconds below 50. The response layer refuses fixes below 50, dead-reckons along the route and releases only when a clean fix rejoins the estimate.

Results (held-out routes, synthetic data): precision 99.3 %, recall 96.1 %, F1 0.977, false alarm rate 0.44 %, 7 of 7 attack types caught; the stealth drift (OSNMA still valid) is caught after 42 s, where a fixed 150 m rule would need about 3.5 minutes. Response layer: worst position error seen by an ordinary screen 8.0 km, after the response layer 29 m; jamming blank screen 100 s without, 0 s with.

Built. Python engine (36 features, 16 rules, scikit-learn, 42 automated tests), Supabase cloud (Postgres, row-level security, realtime, token-authenticated ingest), Next.js console with a 3D street map of Angers, replay of ten recorded scenarios, a narrated attack story, a space-time view, and a live demo at https://trustnav.vercel.app. A receiver adapter for u-blox NMEA/UBX output is written and tested, ready for the first real antenna.

Real drive, no attack. On a public 27.6-minute u-blox ZED-F9K drive recording the rules raise no false alarm, the simulator-trained model must be re-calibrated on the receiver (it alerts on 89 % of the seconds before, on none of the last 8.5 minutes after), and overlaid teleport, drift and jamming are then caught in 0, 35 and 1 s.

Real attacks. Public recordings from Jammertest 2024 in Norway (VAARA dataset, University of Vaasa) were replayed unchanged. A phone was captured by a real spoofer for 26.5 minutes and ended 10.2 km from the truth: the version first published alerted only after 21 min 49 s and refused 5 of 909 captured fixes, because nothing in the spoofed stream was physically impossible. What the spoofer could not hide was time, so a sixteenth rule checks the receiver's time against the host clock: the alert now comes in the same second, all 909 captured fixes are refused, and there is no false alert in the clean periods. Real jamming of a u-blox receiver is alerted in 15 of 20 exposures (1 of 20 by the rules alone).

Limits. The detection table comes from one simulator, so those numbers are an upper bound; the real attacks are two days of one campaign. Nothing is transmitted: our attacks exist only in software and the real ones are recordings made by others. It detects and responds; it cannot remove a jammer from the air.

Next. Plug in a real receiver (about 60 to 250 EUR), record hours of normal drives around Angers, re-calibrate, then run the raw radio recordings of real attacks (Jammertest 2023, TEXBAT) through a software receiver. See ROADMAP.md (section 8).

Team. Group 2, MSc Cybersecurity & Data Science, ESAIP Angers: Karan Deshmukh, Sudheer Reddy Jogi, Niranjan Satheesh Thandassery, Venkata Pavan Kumar Sanam. Tutor: Sofiane Hamrioui.

2 The console, in plain words

The live console is the part a jury member sees first, so this section says what each element means in everyday words. A shorter version is one keypress away inside the console (`?`).

What it shows. Every second, for each van and drone, TrustNav checks whether its satellite position can be believed and gives it a score from 0 (certainly false) to 100 (certainly true). Below 50 it refuses the position and tracks the vehicle by its odometer instead, so nothing downstream acts on a lie.

The map. A ring around each vehicle is its trust score drawn as a gauge: green is trusted, amber has minor doubts, orange is suspicious, pink is untrusted. A hollow marker labelled "refused" is the position the receiver claims and TrustNav is not passing on. A teal shield is where TrustNav believes the vehicle really is, worked out from its odometer along its route. The dashed line between them is the lie, with its length; in the teleport attack it is 8 km. The coloured trail is the last 90 seconds of reported positions.

The panels. The strip at the top left shows the fleet's average trust, how many vehicles are in each band, the open incidents, and how many positions are being refused with the size of the worst lie. The alerts panel at the top right has one card per incident with the last minute of trust, the action taken (verify or isolate) and a tick to acknowledge it. Clicking any vehicle opens its file: first a verdict in plain words, then the evidence (what was measured, and its limit or normal range), then what to do.

The three things that go wrong. *Jamming*: a noise transmitter drowns the satellites and the receiver loses its position; TrustNav keeps an estimate from the odometer, so the screen never goes blank. *Spoofing*: a transmitter imitates the satellites and feeds a false position, either a jump or a slow drift; TrustNav notices the impossible motion, the too-perfect signal or the drift off the road, and refuses the position. *Forgery*: fake Galileo navigation data; Galileo signs its messages (OSNMA), so the signature fails and the forgery is caught within seconds.

Presentation mode (key `P`) makes the camera visit every platform, incidents first, with a caption, for a jury room or a wall screen. **Replay** plays ten recorded attacks with a narrated story. **Results** has the numbers. **Method** explains how it decides and how it differs from existing products.

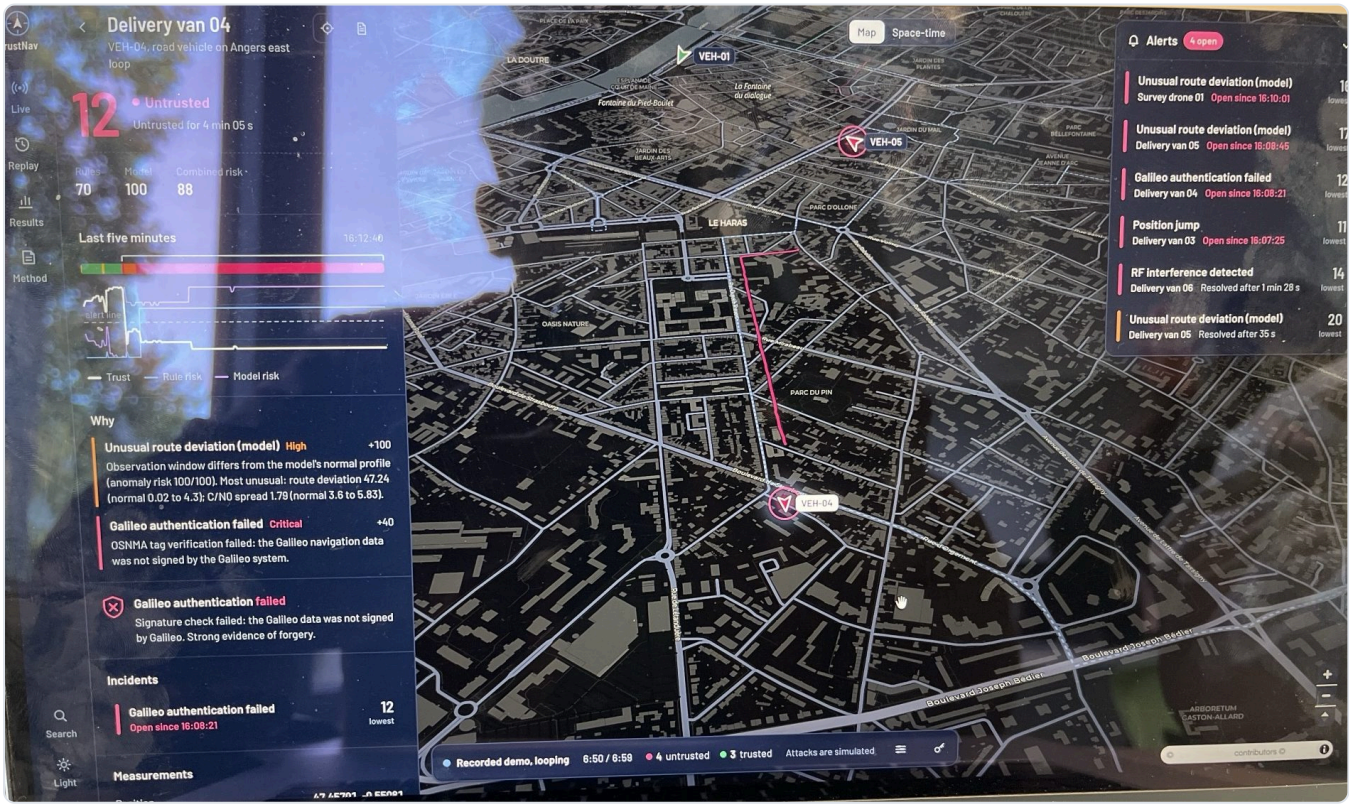


Figure 1. The deployed console (trustnav.vercel.app) photographed on 29 October 2026: Delivery van 04 under a Galileo forgery attack, untrusted for 4 minutes, with the forged track in pink over the real streets of Angers.

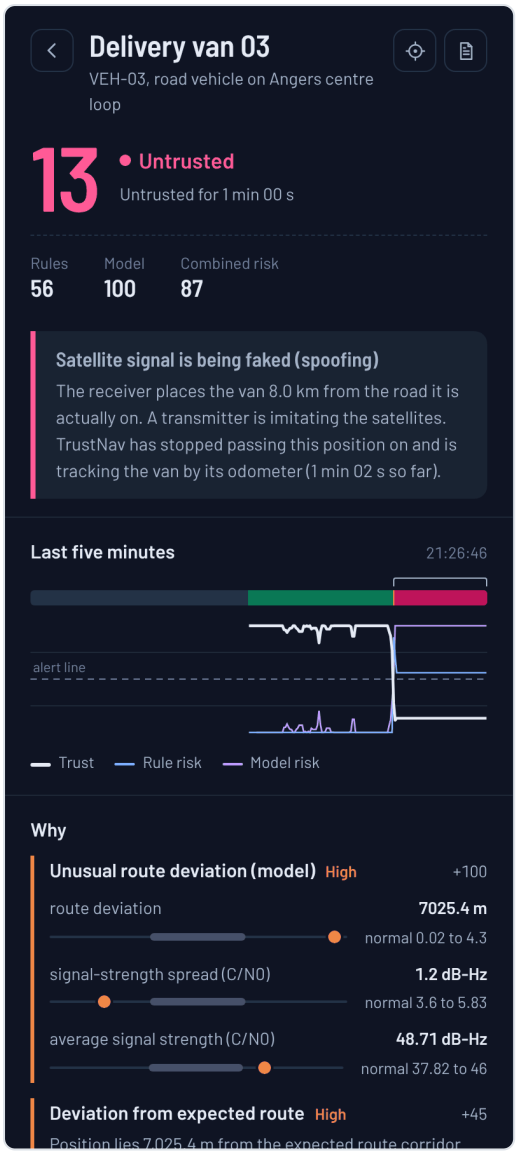


Figure 2. A vehicle's file: the score, the verdict in plain words, the last five minutes, and the evidence.

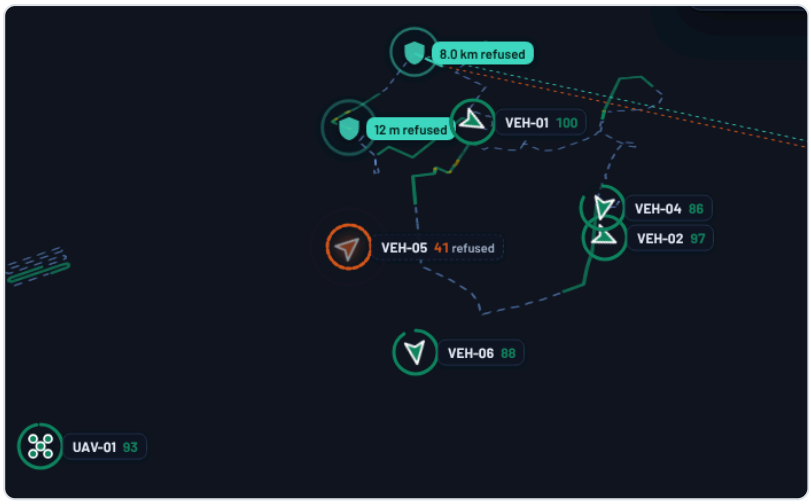


Figure 3. The response layer on the map: hollow refused fixes, teal shields where the vehicles really are, and the dashed lies ("8.0 km refused"). The basemap is simplified in this test render.

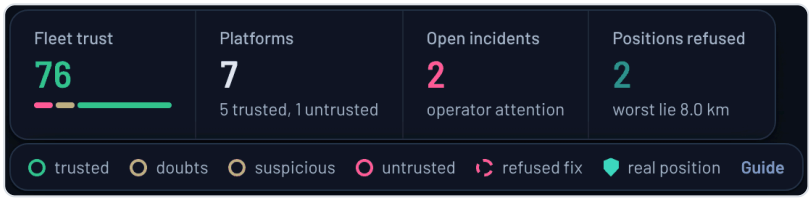


Figure 4. Fleet at a glance.

3 Results

Ten held-out scenarios, 3,800 one-second observations, on routes the model never saw in training. Synthetic data, so an upper bound; replays on real recordings follow.

	Rules only	Model only	Combined (TrustNav)
Precision	100.0 %	97.9 %	**99.3 %**
Recall	27.9 %	90.1 %	**96.1 %**
False alarm rate	0.00 %	1.33 %	**0.44 %**
Mean time to detect	9.0 s	6.57 s	**6.71 s**

7 of 7 attack types are caught; the stealth drift (Galileo signatures still valid) is caught after 42 s, where a fixed 150 m rule would need about 3.5 minutes.

What the response layer prevents. Worst spoofing error an ordinary fleet screen would show: **8,003 m**. Worst error of what TrustNav hands downstream: **29 m**. Jamming blank screen: **100 s** without, **0 s** with.

On recordings of real attacks (Jammertest 2024, Norway; VAARA dataset, University of Vaasa). A phone in a car was captured by a real spoofer for 26.5 minutes: its clock was set back 60 minutes and its position 34 m aside, and when the car drove off the phone stayed where the spoofer had put it, ending 10.2 km from the truth. The version first published alerted after 21 min 49 s and refused 5 of 909 captured fixes. With a sixteenth rule, receiver time checked against the phone's own clock, the alert comes in the same second and **909 of 909** are refused, with no false alert in the clean periods. Real jamming of a u-blox receiver: **15 of 20** exposures alerted. On a clean real drive of 27.6 minutes the rules raise no false alarm.

4 What comes next

Prevention, honestly. No software can stop a jammer or a spoofer from transmitting; only antenna hardware mitigates at the radio level. What software can prevent is the damage: a fleet system acting on a false position. That is what the response layer does today.

Phase 2: real receiver (next two weeks)

Already done: a phone as the receiver, a clean real drive replayed, and recordings of real attacks replayed (section 3).

#	Step	Why	Effort
1	**Plug in a real receiver.** A u-blox USB receiver on the laptop, `python -m trustnav_engine.receiver --port ... --no-model`. The adapter already exists and is tested on synthetic NMEA/UBX.	First real data. Shows the whole chain live: antenna, engine, cloud, dashboard.	1 hour once the device arrives
2	**Record normal drives** with `--jsonl` (walk, tram, car around Angers).	The model's idea of "normal" comes from the simulator. The Stanford study (Kriezis et al., 2025) found thresholds tuned on one receiver failed on another. Real normal data is the only cure.	2 to 3 hours of recording
3	**Re-calibrate** the rules' thresholds and retrain the model on those recordings (`calibrate` script, to write). Report the before/after false alarm rate.	Turns the research prototype into something that is honest about a real device.	1 day
4	**Replay raw radio recordings of attacks** through the same pipeline: the FGI Jammertest 2023 recordings (CC BY 4.0) and TEXBAT (research use) are raw RF, so they need a software receiver (GNSS-SDR) or a receiver fed by an SDR replay to produce observables; the Mendeley receiver-observable dataset can be used directly.	More attack types and more receivers than the two devices replayed so far, still without transmitting anything.	2 to 4 days

Later

A vehicle's own odometer as the inertial input; a second antenna on the same vehicle to catch synchronous spoofing; real Galileo OSNMA from raw pages; map matching for vehicles without an assigned route; alerts by e-mail or SMS; drone integration (PX4); a Raspberry Pi edge box.

Links

Live demo: https://trustnav.vercel.app · Source and full documentation: https://github.com/domail2karandeshmukh-cyber/trustnav

5 Team and contributions

Group 2, MSc Cybersecurity and Data Science, ESAIP Angers. Tutor: Sofiane Hamrioui.

Member	Role	Responsible for
Karan Deshmukh	Project lead	Overall design and architecture. The detection engine (simulator, features, rules, anomaly model, fusion, response layer, receiver adapter). The cloud side (database schema, row-level security, ingest function, demo feed). The console (map, replay, story, space-time view, presentation mode). The documentation and the communication with the tutor and the jury.
Niranjan Satheesh Thandassery	Evaluation and validation	The evaluation protocol (train, validation and held-out splits), the recorded attack scenarios and the results tables. Testing of the console and the end-to-end checks. The Phase 2 hardware plan: receiver and antenna choice, recording of normal drives, calibration.
Sudheer Reddy Jogi	Research and comparison	The market and literature study behind the comparison with existing solutions (receiver flags, fixed probes, anti-jamming hardware, fleet telematics, OSNMA, recent papers). Review of the documentation. Preparation of the oral presentation.
Venkata Pavan Kumar Sanam	Testing and presentation support	Functional testing of the live demo and the recorded scenarios, with issue reports. The safety, legal and ethics notes. The demo rehearsal and the presentation material for the oral examination.

How the team works. One repository on GitHub with every change reviewed by the lead; the live demo is redeployed automatically on every change; three tutor meetings (the first in the last week of October, the second and third before the oral examination on 13 November 2026); the first real receiver is demonstrated at the third meeting.